

CONTEXT DOCUMENT · FOR CLAUDE (CODE, DESKTOP APP, COWORK)

Crucible

What the software is, how its data is shaped, which connector tools exist, and how to act inside it well.

Give this file to Claude at the start of a session, or keep it in the project folder as reference. It matches Crucible 0.6 and connector tools version 0.6 (September 2026).

Crucible is proprietary software by Armaan. This document describes behaviour, not source code.

1 · What Crucible is

Crucible is a local, offline production tracker for one developer or 3D artist: a project has departments, departments hold work items, work items can nest (asset → pipeline stages), and a deterministic planner places open work on days according to capacity. It also tracks bugs, logged time, progress and a per-item history, and reports health and a forecast. There is no server, no account and no cloud. All data is JSON on the user's PC.

Your role through the connector. You are a helping hand: create and organise projects, file work and bugs where they belong, keep estimates and time honest, and read the plan back to the user. Crucible's own rules (planner, health, next steps) are deterministic and stay in charge; you feed them good data and explain their output.

The core loop

project → departments (production structure) → work items → child items / pipeline stages → schedule (planner) → daily work → time and progress → comments and screenshots → history → forecast / replan → reports.

Vocabulary

TERM	MEANING
project	One folder of data with its own start date, duration (14–365 days, "sprint"), hours/day, working days, departments and items. Several projects can exist; one is <i>current</i> for the connector. Days are numbered from 1 = start date.
department	A production area (Modeling, Texturing, Gameplay, QA ...). Comes from a catalog of ~40 or is custom. <i>In use or parked</i> ; parked departments keep items but are excluded from planning and health. Each has an editable <i>pipeline</i> (ordered stages with estimates).
work item	The universal record. Type: TASK, ASSET, DESIGN, MODELING, GAMEPLAY, IMPROVEMENT, BUG, POLISH, QA, FUTURE. Has department, status, priority, optional class S/A/B/C/D, severity (bugs), estimate, logged hours, remaining, progress 0–100, planned start/end day, parent, dependencies, tags, checklist, history.
asset with pipeline	An item whose children are the department's pipeline stages (Reference → Modeling → UV → Baking → ...). The parent's progress and hours roll up from the stages. Create with <code>create_item type=ASSET</code> or <code>pipeline=true</code> .
status	BACKLOG (known, unscheduled) · PLANNED · TODO (scheduled, not started) · IN_PROGRESS · BLOCKED · DONE · CUT. Only non-BACKLOG, non-DONE items are planned.
priority	MUST_HAVE · NICE_TO_HAVE · CUT. MUST_HAVE work is what the forecast and deficit are measured against.
severity	Bugs only: CRITICAL · HIGH · MEDIUM · LOW. CRITICAL/HIGH bugs become MUST_HAVE automatically.
planner	Deterministic scheduler: pins → dependencies → critical path → class → priority → deadlines → department order. Real capacity = working days × hours/day. Reports conflicts, deficit, critical path, forecast day and variance. "open" mode replans open items from today; "all" replans everything from day 1.

health	ON_TRACK · AT_RISK · LATE · CRITICAL with reasons; forecast day vs original end.
next steps	Rule-based list of what to do next (blocker → continue → next stage → start → add department → replan). The first entry is <i>the</i> next step.
time log	Hours are logged as entries with a note; the item's logged hours is their sum. Progress is separate and manual.

2 · The connector tools

All tools take an optional `project` id; without it they act on the current project. Department arguments accept the id or the name (case-insensitive). Every result is compact JSON. Errors are plain sentences that say what to do instead (for example the list of available departments).

Read

TOOL	USE IT FOR
<code>list_projects</code>	Which projects exist, which is current, the data folder. Call first when unsure.
<code>get_project</code>	Summary: dates, day, capacity, remaining, deficit, health, forecast, departments with progress, item counts.
<code>search_items</code>	Find items: text (title, description, repro, location, tags, id, department) + type, plan state (open/all/unscheduled/planned/today/overdue/in_progress/blocked/done), priority, severity, department. Default plan state is open ; pass <code>plan=all</code> to include DONE/CUT.
<code>get_item</code>	One item in full: fields, children, dependencies, checklist, history thread.
<code>health</code>	The Dashboard numbers and reasons, overdue items, scope pressure.
<code>next_steps</code>	The next-step list with reasons.
<code>today</code>	Today's ranked actionable work with WHY reasons.
<code>recommend_departments</code>	Departments not yet in use that the description rules or the pipeline suggest, with reasons.

Write

TOOL	USE IT FOR
<code>create_project</code>	New project: name, description (drives the recommended departments), days, hoursPerDay, workingDays (0 = Sunday), startDate, profile, or explicit catalog department ids; <code>seedTemplates</code> (default true) puts one pipeline template item per department; <code>firstAssets</code> creates assets with the full pipeline. Becomes current.
<code>set_current_project</code>	Point the connector at another project.
<code>add_department</code>	Put a catalog department (or a custom name) in use; returns the plan impact (+hours, +days, forecast before → after). Extends production assets with a stage by default.
<code>create_item</code>	Task / asset / bug. Needs a department (or a parentId, whose department is used). Bugs: severity, repro, location; they land on the current day. Other items go to the backlog unless <code>day</code> is given. <code>type=ASSET</code> or <code>pipeline=true</code> adds the pipeline stages as children.
<code>update_item</code>	Change title, description, repro, location, tags, estimate, priority, severity, department, planned start/end. Only passed fields change; each change is logged.
<code>set_status</code>	Any status. Crucible may refuse (for example DONE with unfinished required steps) and says why.
<code>set_progress</code>	0–100 with a note.
<code>log_time</code>	Hours with a note (negative hours correct an earlier entry). Shows in the Time log screen.

<code>add_comment</code>	Dated comment in the item history.
<code>plan</code>	Run and apply the scheduler (open or all). Returns placed count, forecast, deficit, conflicts.

There is deliberately no delete. Everything you write is marked in the history with author "Claude"; items you create carry the tag `via-claude`. The Crucible window reloads within about two seconds after you write.

3 · How to act well

1. **Look before you write.** `list_projects` when the project is unclear; `get_project` for the department list; `search_items` before creating, so you do not duplicate.
2. **File things where they belong.** A bug found on an asset goes under that asset (`parentId`). A stage of an asset is a child, not a top-level task. Pick the department that does the work, not the one that noticed it.
3. **Estimates are hours of work, not calendar time.** Give every task an estimate; leave bugs at the default unless the user knows.
4. **Severity is about impact, priority is about scope.** Crashes, blockers, data loss → CRITICAL/HIGH. Visual polish → LOW. Let the severity set the priority unless told otherwise.
5. **Backlog vs day.** Capture new work into the backlog unless the user says when it happens. Bugs go on today. Do not scatter items over days yourself; run `plan` and let Crucible place them.
6. **Use** `plan mode=open` after adding a batch of work. Only use `mode=all` when the user asks to replan everything.
7. **Report back with ids and numbers.** After a batch: what was created (titles, ids, departments), and the plan result (forecast day, variance, deficit, conflicts). Say plainly when the forecast is past the end date.
8. **Never invent departments** when one exists under a slightly different name; the error message lists the real ones. Use `add_department` only when the work truly needs a new area, and tell the user the impact it returned.
9. **Keep notes short and factual** in time entries and comments; they are read by the user later in the History tab and Time log.
10. **Do not touch what the user did not ask about.** No mass status changes, no reprioritising, no reordering, unless asked.

Reading playtest or meeting notes into bugs

One bug per distinct problem. Title = what is wrong in one line. `repro` = steps. `location` = where (level, screen, asset). Severity from impact. Department = who fixes it (Gameplay for logic, Level for layout, Art/Modeling for asset faults, UI for interface, Audio, QA only for test tasks). If the note names an asset that exists, use `search_items` to find it and set `parentId`.

4 · Prompt templates

New project with departments and first work

```
Use the Crucible connector.
Create a new project:
- name: Temple Props Pack
- description: Production-ready Unreal Engine 5 medieval props: high-poly modeling, retopology,
UVs, baking, texturing, materials, LODs, collision, engine integration and QA.
- 45 days starting today, 6 hours/day, Monday to Friday.
Let Crucible recommend the departments from the description and keep its templates.
Then create these assets with the full pipeline in the modeling/environment department: Altar,
Brazier, Stone Bench, Iron Gate, Broken Column.
Add one task "Reference board and style sheet" (4 h, MUST_HAVE) in the concept or design
department.
Run the planner in open mode and tell me the forecast day, variance, capacity vs remaining hours,
and any conflicts.
List everything you created with ids.
```

Update tasks and log a day's work

```
Use the Crucible connector on the current project.
1. Find the item "Altar" and its stages.
2. Mark the Reference stage DONE and set the Modeling stage IN_PROGRESS with 60% progress.
3. Log 3.5 hours on the Modeling stage with the note "blockout + primary forms".
4. Raise the estimate of the UV stage to 3 hours.
5. Add a comment on Altar: "Client wants more wear on the top surface."
Then show me today's ranked work and the current health.
```

Playtest notes into bugs

```
Use the Crucible connector on the current project. Turn these notes into bugs, one per problem,
with severity, repro steps, location and the right department. Attach a bug to the asset it
belongs to when that asset exists (search first). Then give me a table of what you filed with ids
and a count by severity.
```

Notes:

- fell through the floor next to the altar after jumping twice
- torch flicker is way too fast, looks like strobe
- gate opens the wrong way and clips through the wall
- menu font is unreadable at 1080p
- game froze for 2 s when the second wave spawned

Daily check-in

```
Use the Crucible connector. What is my next step, what is today's ranked work, and is the project
on track? If the forecast is past the end date, explain the reasons Crucible gives and which
nice-to-have items it suggests deferring. Do not change anything.
```

Add a department when the pipeline needs it

Use the Crucible connector. Which departments does Crucible recommend adding next and why? Add the first one it marks as suggested next, extend the assets with its stage, and tell me the planner impact (hours added, days added, forecast before and after).

5 · Setup reminders

- **Claude Code:** `claude mcp add --scope user crucible -- node "<folder>\crucible-mcp.cjs"` once; every new chat then has the tools.
- **Claude desktop app / Cowork:** add `{"mcpServers":{"crucible":{"command":"node","args":["<folder>\\crucible-mcp.cjs"]}}}` to `claude_desktop_config.json` (Settings → Developer → Edit config), restart the app. The exact snippet with the real path is in Crucible → Project Settings → Claude connector.
- Needs Node.js 18 or newer. Data folder: `%APPDATA%\dev.armaan.crucible\projects\<id>\project.json`. Override with `--data <folder>`.
- If a tool says "No project stored yet", open Crucible once or call `create_project`.

Crucible 0.6 · © 2026 Armaan · All rights reserved.